

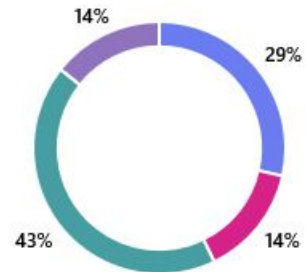
L09: Engine and Game architecture

Chris Carvelli

[GameProgramming25] Advanced Topics poll



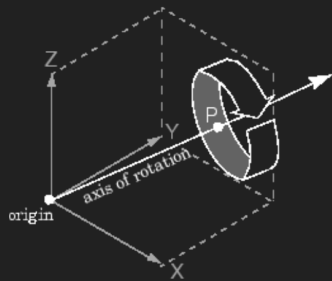
● Advanced Memory Management	4
● Randomness and PCG	2
● Data structures for Games	6
● One extra supervision lecture	2



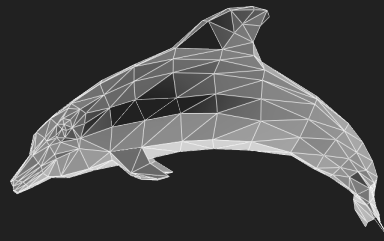
ITU Game Consoles Workshop



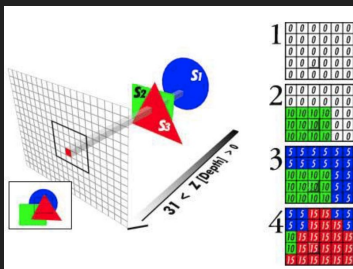
Recap



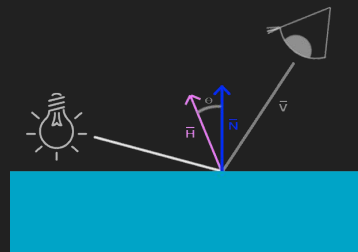
rotations



meshes



3D rendering



lighting models

An isometric illustration of a game architecture. It features a central grey stone tower with battlements and a crown on top. To the left, a yellow and orange structure with a long, steep orange staircase leads up to a platform. To the right, another yellow and orange structure with a similar staircase leads down. A small white figure with a pointed hat stands on a yellow platform on the right. Several black figures with white circles on their heads are positioned on various platforms and the central tower. The background is a dark blue gradient with small orange diamond shapes. A semi-transparent black rectangle with white text is centered over the image.

L09 - Engine and Game architecture

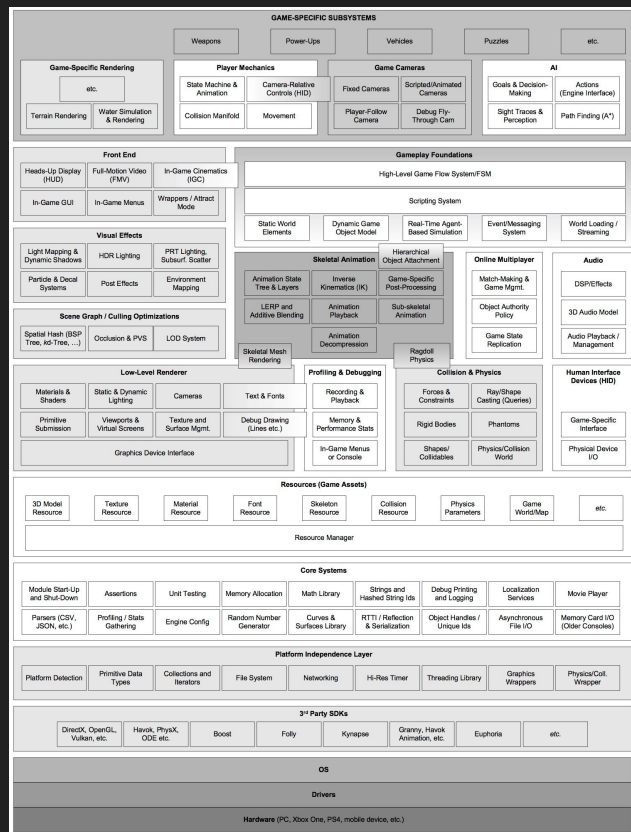
Agenda

1. Anatomy of a game
2. "Engine" interface
3. Quick tips

Agenda

1. Anatomy of a game
2. "Engine" interface
3. Quick tips

Game engines architecture overview

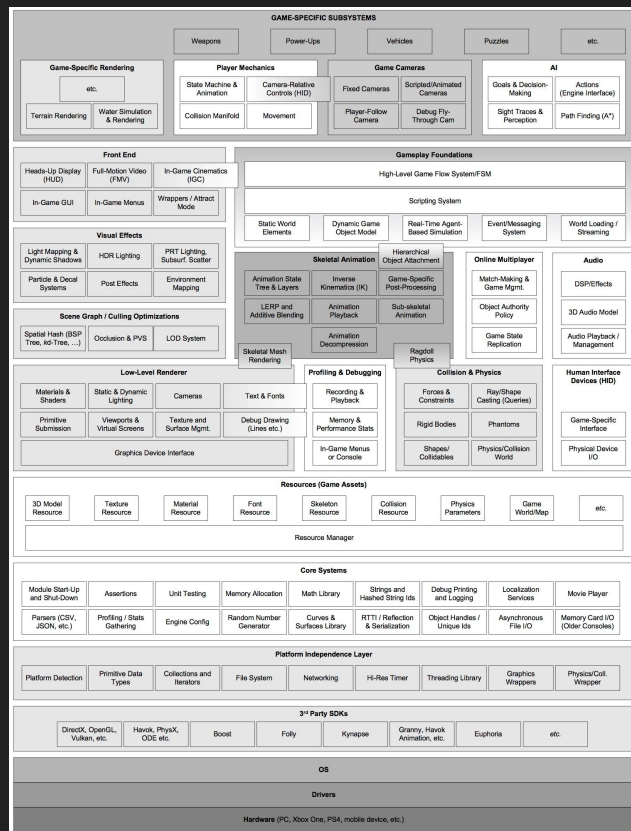


Game-specific subsystem

Game engine

OS, hardware, platform abstraction

Through exercises' lenses



Exercise code

itu (custom utilities)

assimp

box2d

stb

glm

imgui

SDL_mixer

SDL_ttf

SDL

"Engine" zoom in

Exercise code

itu_entity_storage.hpp

itu_resource_storage.hpp

itu_sys_transform.hpp

itu_lib_debug_ui.hpp

itu_lib_fileutils.hpp

itu_lib_render.hpp

itu_lib_sprite.hpp

itu_sys_physics.hpp

itu_lib_imgui.hpp

itu_lib_box2d.hpp

itu_lib_math.hpp

itu_lib_overlaps.hpp

itu_lib_render3d.hpp

itu_lib_engine.hpp

itu_common.hpp

SDL & libs

Engine<>game interface layer

Exercise code

itu_entity_storage.hpp

itu_resource_storage.hpp

itu_lib_fileutils.hpp

itu_sys_transform.hpp

itu_lib_debug_ui.hpp

itu_lib_fileutils.hpp

itu_lib_render.hpp

itu_lib_sprite.hpp

itu_sys_physics.hpp

itu_lib_imgui.hpp

itu_lib_box2d.hpp

itu_lib_math.hpp

itu_lib_overlaps.hpp

itu_lib_render3d.hpp

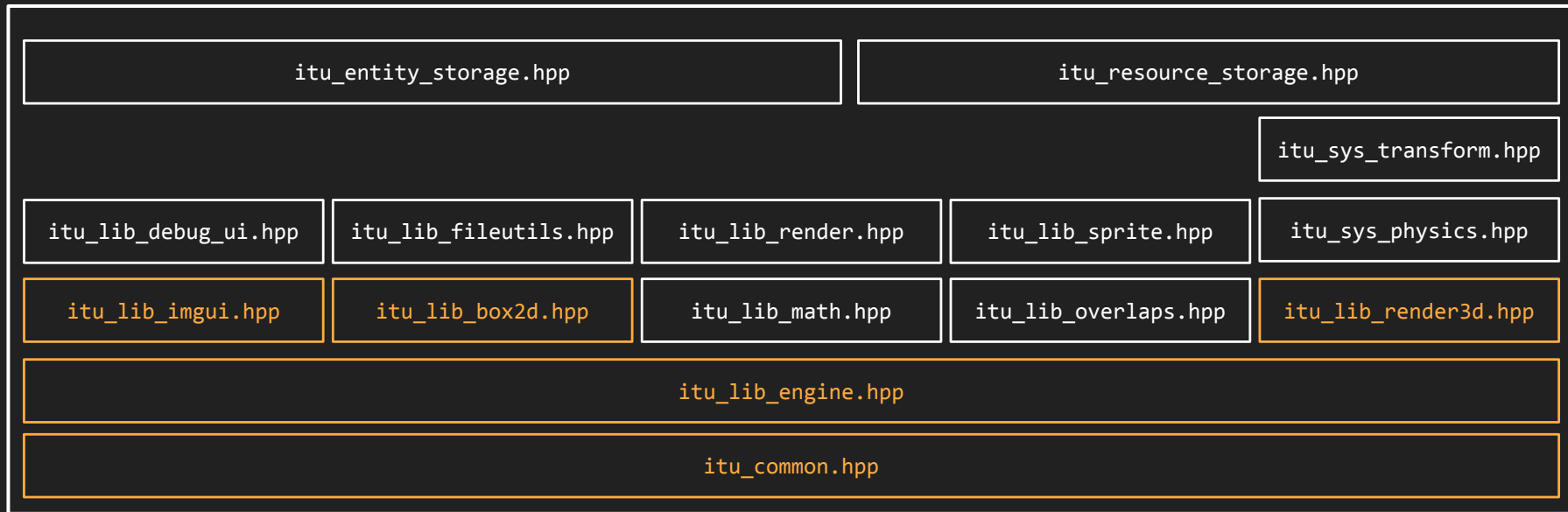
itu_lib_engine.hpp

itu_common.hpp

SDL & libs

Engine<>platform interface layer

Exercise code



SDL & libs

Outliers

Exercise code

itu_entity_storage.hpp

itu_resource_storage.hpp

itu_sys_transform.hpp

itu_lib_debug_ui.hpp

itu_lib_fileutils.hpp

itu_lib_render.hpp

itu_lib_sprite.hpp

itu_sys_physics.hpp

itu_lib_imgui.hpp

itu_lib_box2d.hpp

itu_lib_math.hpp

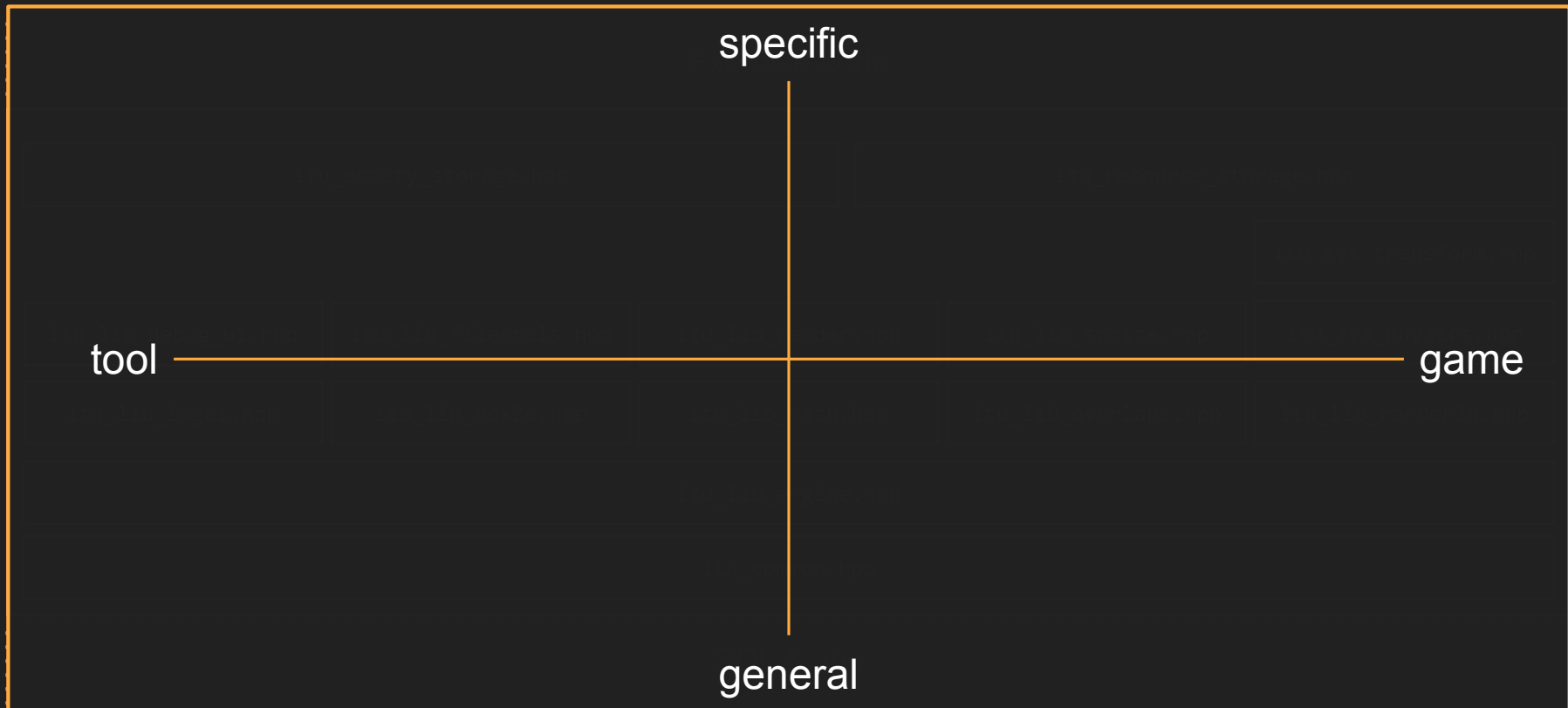
itu_lib_overlaps.hpp

itu_lib_render3d.hpp

itu_lib_engine.hpp

itu_common.hpp

SDL & libs



Tightening up

1. prevent layers from bleeding into each other
2. sandbox 3rd party libraries
3. make a clearer division between tool code, debug code and game code

Agenda

1. Anatomy of a game
2. "Engine" interface
3. Quick tips

Interaction between game and engine code

Usually, gameplay code deals only with the most high-level concepts that we develop

Entities

stuff living in the game world

- the player
- menu buttons
- static objects
- audio emitters
- mesh renderers
- UI label

Resources

assets used by entities

- meshes
- textures
- audio files
- scenes
- fonts
- strings

Scenes

collections of entities

- level N
- main menu
- pause menu overlay

Entities

Systems vs entities

low level systems reason on SOFTWARE
objects/structs

godot systems/servers

nice to allow final developer to interact directly
with systems if they so choose

entities as high level objects that populate a
scene

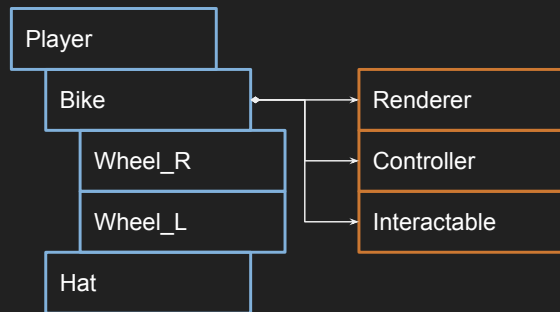
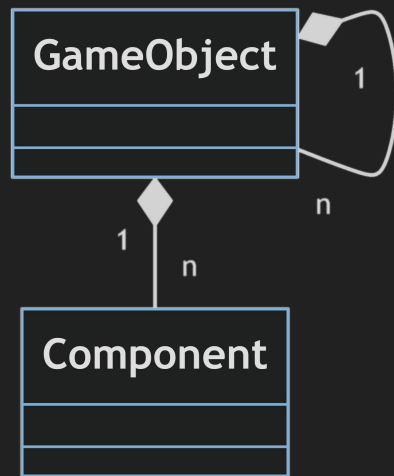
entities most of the time wrap around low level
systems

Types of entity manager systems

Entity is an overloaded term, everyone calls them differently (also due for commercial/IP reasons)

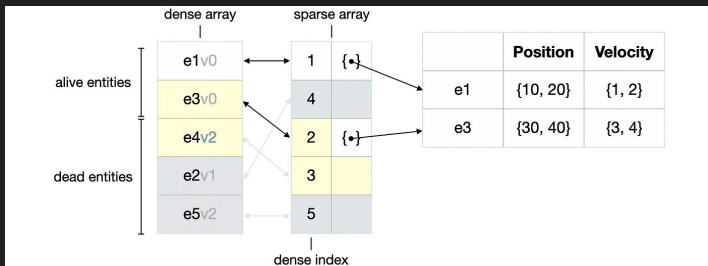
Object/Component overview

- Common implementation starting from the 2000s
- Pros
 - handles transform hierarchies implicitly (with the problems we discussed)
 - very easy to implement
 - easy to use and understand
- Cons
 - potentially poor performance



ECS overview

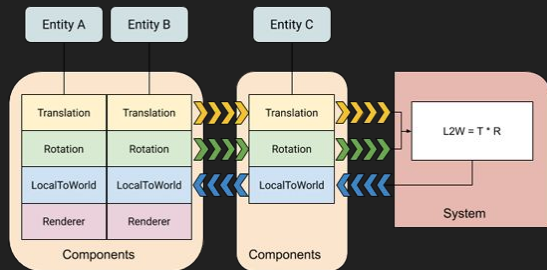
- Entity Component System
- first known game application: Thief the Dark Project (1998!)
- tries to optimize memory layout for fast access, which can mean many things



sparse array ECS

	Entity 1	Entity 2	Entity 3
Component 1	[Data]		
Component 2	[Data]	[Data]	[Data]
Component 3		[Data]	[Data]

a bad ecs example!



archetype-based ECS

Resources

Resources for a programmer

- data needed by the entities to do their thing
- we need efficient representations for them serialized (on disk) and at runtime
- we need an efficient and convenient way to reference said resources

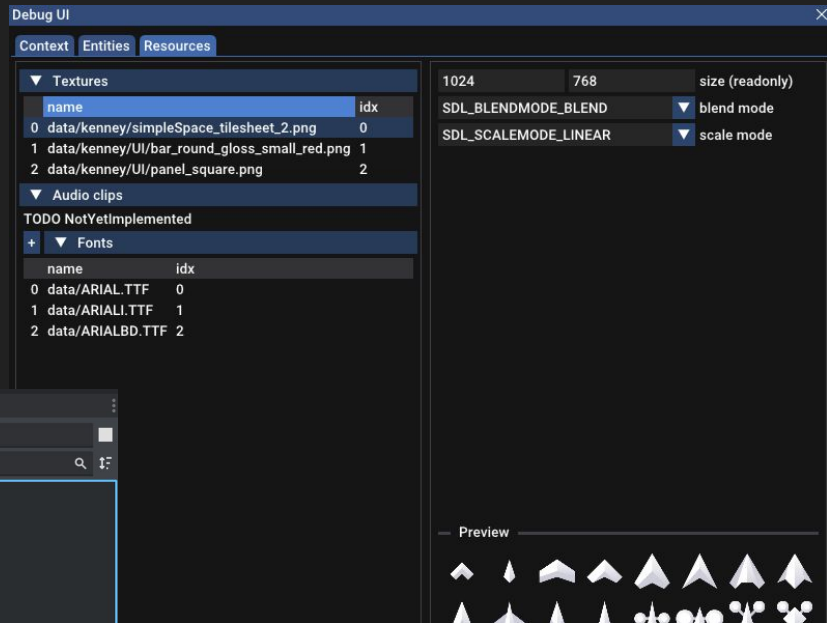
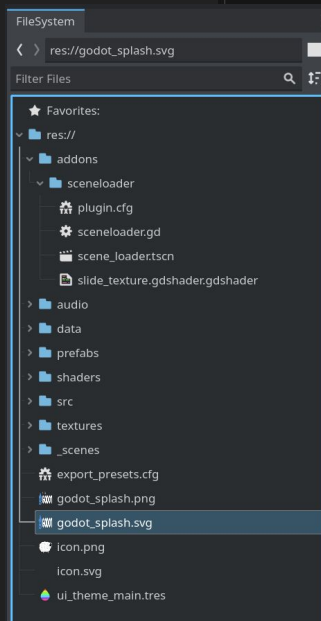
```
ITU_IdModel3D truck = itu_sys_rstorage_model3d_load(  
    context,  
    "truck.obj"  
);  
  
// BAD! (reference with magic number)  
Model3D* m = itu_sys_rstorage_model3d_get_ptr(0);  
  
// a bit better (reference with programmatical ID)  
Model3D* m = itu_sys_rstorage_model3d_get_ptr(truck);  
  
// common shortcut, not great (reference with string path)  
Model3D* m = itu_sys_rstorage_model3d_get_ptr("truck.obj");  
  
// best (reference with string path)  
ITU_IdModel3D id = read_from_file();  
Model3D* m = itu_sys_rstorage_model3d_get_ptr(id);
```


Resources for everybody else

Resources is the single way EVERYONE on the team will interface with the game, no matter their discipline! It's even more important how non-technical users interact with them.

A commonly used trick is to use the file-system as "database" of assets, since everyone is already familiar with it.

- NOT the most effective way for big projects
- it's just an interface, there is still a dedicated system underneath it

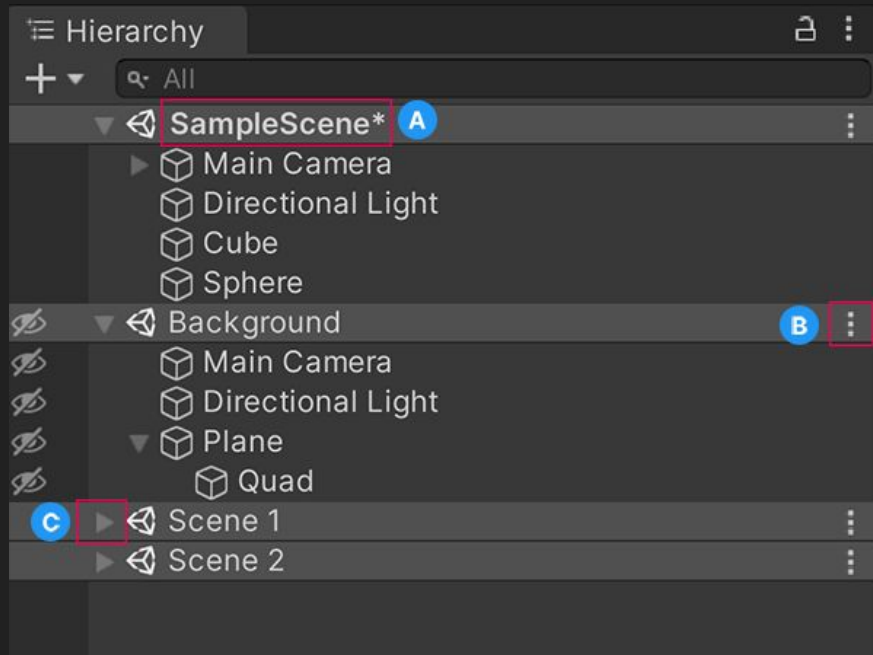


Scenes

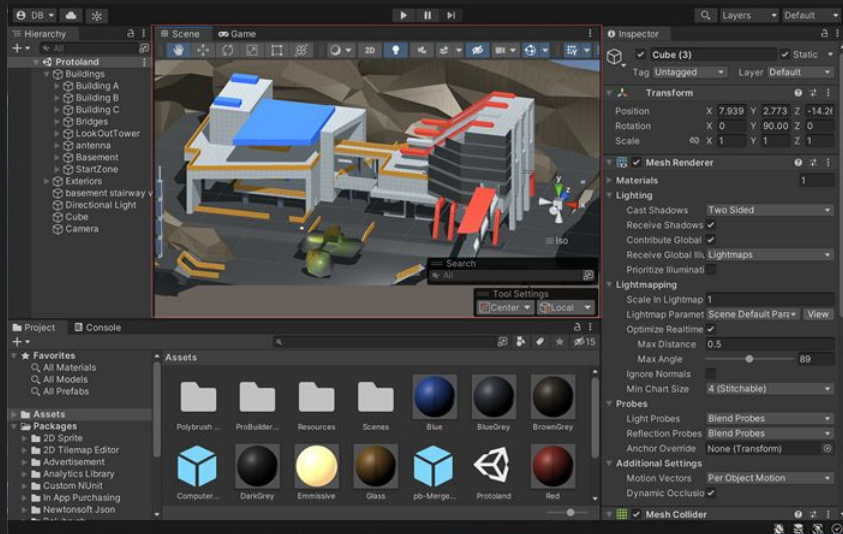
Scenes

Every game above a tech demo will be composed on multiple discrete moments, with their different entities, game logics, resources and so on.

The term "scene" usually refers to both the serialized resource that contains everything present in this section of the game AND the runtime collection of entities, resources and runtime data.



Scenes

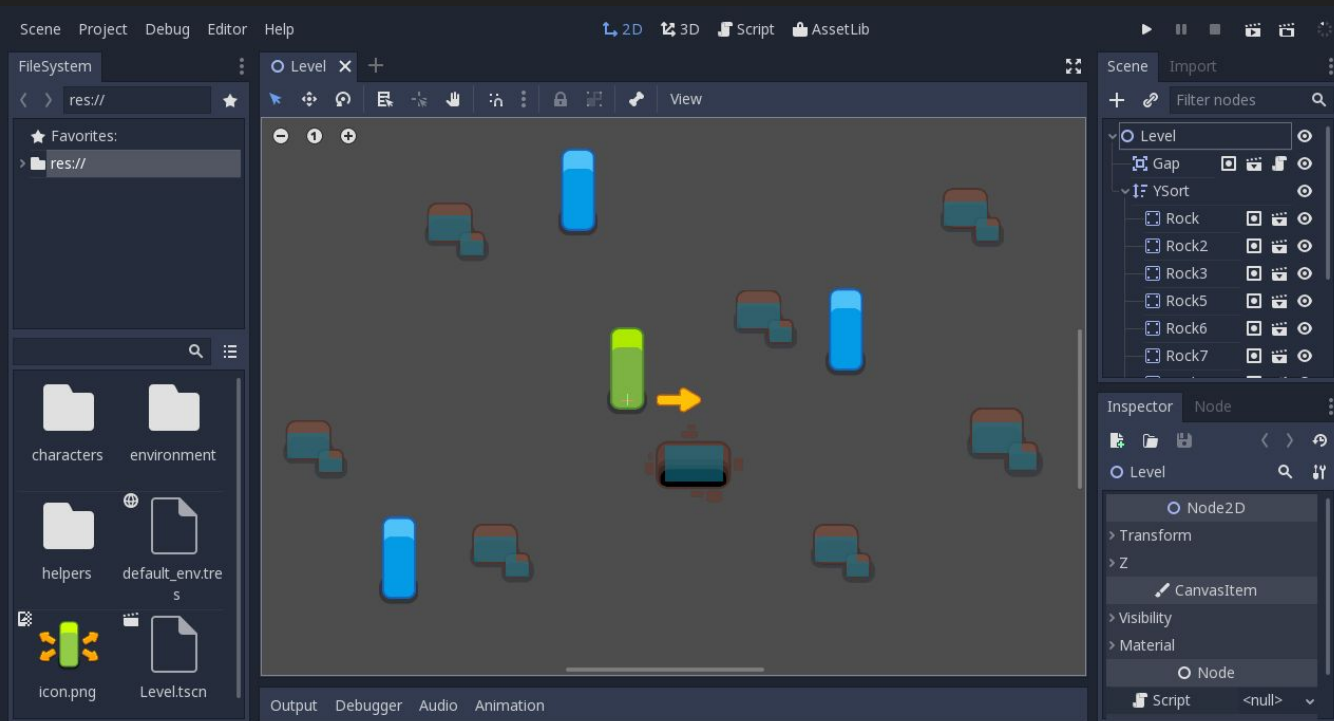


```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!29 &1
OcclusionCullingSettings:
  m_ObjectHideFlags: 0
  serializedVersion: 2
  m_OcclusionBakeSettings:
    smallestOccluder: 5
    smallestHole: 0.25
    backfaceThreshold: 100
  m_SceneGUID: 00000000000000000000000000000000
  m_OcclusionCullingData: {fileID: 0}
--- !u!104 &2
RenderSettings:
  m_ObjectHideFlags: 0
```

[...]

```
m_MeshOptimized: 0
--- !u!1 &1039743499
GameObject:
  m_ObjectHideFlags: 0
  m_PrefabParentObject: {fileID: 0}
  m_PrefabInternal: {fileID: 0}
  serializedVersion: 5
  m_Component:
  - component: {fileID: 1039743500}
  - component: {fileID: 1039743501}
  - component: {fileID: 1039743502}
  - component: {fileID: 1039743503}
  - component: {fileID: 1039743504}
  - component: {fileID: 1039743505}
  m_Layer: 0
  m_Name: patricia
  m_TagString: Untagged
  m_Icon: {fileID: 0}
  m_NavMeshLayer: 0
  m_StaticEditorFlags: 0
  m_IsActive: 1
--- !u!4 &1039743500
Transform:
  m_ObjectHideFlags: 0
  m_PrefabParentObject: {fileID: 0}
```

Godot example



CLASS REFERENCE

All classes

⊕ Globals

⊕ Nodes

⊕ Resources

⊕ Other objects

⊕ Editor-only

⊕ Variant types

PhysicsServer3D

Inherits: [Object](#)

Inherited By: [PhysicsServer3DExtension](#)

A server interface for low-level 3D physics access.

void	area_add_shape (area: RID, shape: RID, transform: Transform3D => Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0), disabled: bool = false)
void	area_attach_object_instance_id (area: RID, id: int)
void	area_clear_shapes (area: RID)
RID	area_create0
int	area_get_collision_layer (area: RID) const
int	area_get_collision_mask (area: RID) const
int	area_get_object_instance_id (area: RID) const
Variant	area_get_param (area: RID, param: AreaParameter) const
RID	area_get_shape (area: RID, shape_idx: int) const
int	area_get_shape_count (area: RID) const
Transform3D	area_get_shape_transform (area: RID, shape_idx: int) const
RID	area_get_space (area: RID) const
Transform3D	area_get_transform (area: RID) const

Godot example

```
class_name NodeInterface
extends Node

@export var starting_velocity: Vector2 = Vector2(0, 0)
@export var acceleration      : Vector2 = Vector2(0, 1)
@export var num_bodies : int = 10

func _ready():
    for i in num_bodies:
        var new_node : RigidBody2D = RigidBody2D.new()
        new_node.add_constant_force(starting_velocity)
        add_child(new_node)

func _process(delta):
    for i in get_child_count():
        var child = get_child(i)
        child.add_constant_force(acceleration * delta)
```

```
class_name ServerInterface
extends Node

@export var starting_velocity: Vector2 = Vector2(0, 0)
@export var acceleration      : Vector2 = Vector2(0, 1)
@export var num_bodies : int = 10

var bodies : Array[RID]

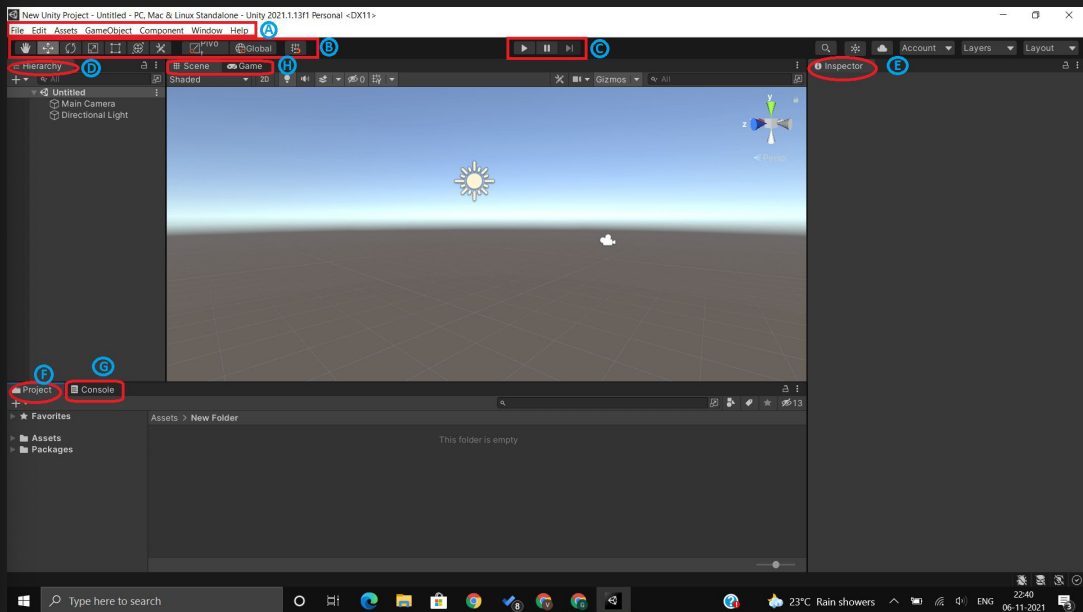
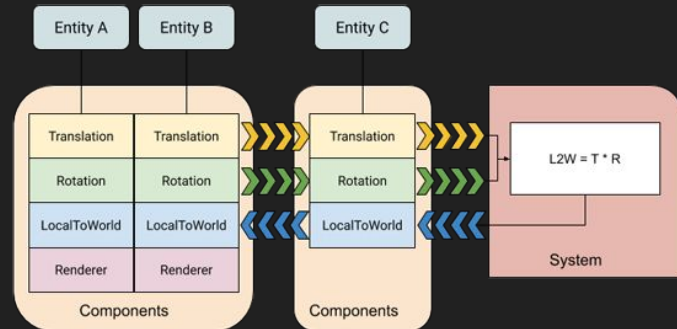
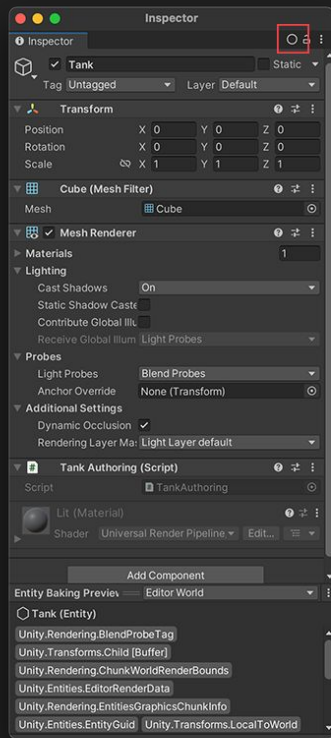
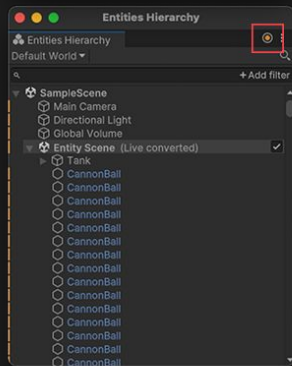
func _ready():
    bodies.resize(num_bodies)
    for i in num_bodies:
        var new_body : RID = PhysicsServer2D.body_create()
        PhysicsServer2D.body_add_constant_force(
            new_body,
            starting_velocity
        )
        bodies[i] = new_body

func _process(delta):
    for i in num_bodies:
        PhysicsServer2D.body_add_constant_force(
            bodies[i],
            acceleration * delta
        )
```

The screenshot shows the Unity Hierarchy panel with the following structure:

- Entity Scene (Live converted)
 - Tank
 - Cannonball

The 'Add filter' button is located in the top right corner of the panel.



Agenda

1. Anatomy of a game
2. "Engine" interface
3. Quick tips

Inversion of control

```
// my_client_game.cpp
int main(void)
{
    engine_init();
    my_game_init();
    while(true)
    {
        engine_update_pre();
        my_game_update();
        engine_update_post();
    }
}
```

```
// engine.cpp (game programmer does not touch this)
extern void game_init();
extern void game_update();
```

```
int main(void)
{
    // engine initialization
    game_init();
    while(true)
    {
        // engine update
        game_update();
        // more engine update
    }
}
```

```
// game.cpp
void game_init() { }
void game_update() { }
```

Static singleton VS dependency injection

```
// system's context stored as global static variable
ITU_EntityStorageContext ctx_estorage;

// part of the context is injected, part is singleton
void itu_sys_estorage_systems_update(SDLContext* context)
{
    for(int i = 0; i < ctx_estorage.systems_count; ++i)
    {
        // static global context
    }
}
```

```
// all context is injected
void itu_sys_estorage_systems_update(
    ContextGlobal* context_global,
    ContextEntities context_entities
)
{
    for(int i = 0; i < context_entities.systems_count; ++i)
    {
        // static global context
    }
}
```

Different interpretations of the same data

```
// system's context stored as global static variable
ITU_EntityStorageContext ctx_estorage;

// part of the context is injected, part is singleton
void itu_sys_estorage_systems_update(SDLContext* context)
{
    for(int i = 0; i < ctx_estorage.systems_count; ++i)
    {
        // static global context
    }
}
```

```
// part of the context is injected, part is singleton
void itu_sys_estorage_systems_update(
    SDLContext* context_global,
    ITU_EntityStorageContext context_entities
)
{
    for(int i = 0; i < context_entities.systems_count; ++i)
    {
        // static global context
    }
}
```

Sandboxing of 3rd party libraries

```
// itu_lib_imgui.hpp
void itu_lib_imgui_setup(Context* context);
bool itu_lib_imgui_process_sdl_event(SDL_Event* event);
void itu_lib_imgui_frame_begin();
void itu_lib_imgui_frame_end(Context* context);
```

```
// game.cpp
ImGui::Begin("Debug UI", &context.debug_ui_show, 0);
{
    ImGui::Text("Timing");
    ImGui::LabelText("work", "%6.3f ms/f", work);
    ImGui::LabelText("tot", "%6.3f ms/f", tot));
    ImGui::LabelText("steps", "%d", steps);
}
ImGui::End();
```

```
// itu_lib_ui.hpp
void itu_lib_ui_setup(Context* context);
bool itu_lib_ui_process_sdl_event(SDL_Event* event);
void itu_lib_ui_frame_begin();
void itu_lib_ui_frame_end(Context* context);
// out API may be almost identical to what we are wrapping,
// or very different
bool itu_lib_ui_begin(Context* context, char* title);
void itu_lib_ui_end(Context* context);
void itu_lib_ui_text(char* title, ...);
void itu_lib_ui_label_text(char* label, char* title, ...);
```

```
// game.cpp
itu_lib_ui_begin(*context, "Debug UI");
{
    itu_lib_ui_text("Timing");
    itu_lib_ui_text("work", "%6.3f ms/f", work);
    itu_lib_ui_label_text("tot", "%6.3f ms/f", tot));
    itu_lib_ui_label_text("steps", "%d", steps);
}
itu_lib_ui_end();
```

Zero vs error initialization

```
struct MyData;
struct MyData
{
    float width;
    float height;
    MyData* next;
};

// naive zero-initialization
int main(void)
{
    MyData data = { 0 };

    printf("%f\n", data.width);           // prints 0
    // printf("%f\n", data.next->width); // Segmentation Fault

    // -program crashes
    // -debugger shows 0 everywhere, unclear
    // if that's intentional or an error

    return 0;
}
```

```
// zero initialization with pointer to empty
static MyData MYDATA_EMPTY = { 0, 0, &MYDATA_EMPTY };
int main(void)
{
    MyData data = MYDATA_EMPTY;
    printf("%f\n", data.width);           // prints 0
    printf("%f\n", data.next->width);     // prints 0

    // +program does not crash
    // -difficult to spot eventual errors
    return 0;
}

// error initialization
int main(void)
{
    MyData data;
    memset(&data, 0xdf, sizeof(MyData));

    printf("%f\n", data.width);           // prints garbage
    //printf("%f\n", data.next->width); // Segmentation Fault

    // -program crashes
    // +debugger shows 0xdf everywhere, easy to spot error
}
```

Reasonable defaults

```
// no tradeoff here, avoid defines where possible
#define TEXTURE_PIXELS_PER_UNIT 128
#define CAMERA_PIXELS_PER_UNIT 32

#define ENABLE_DIAGNOSTICS

#define TARGET_FRAMERATE_NS      (SECONDS(1) / 60)

#define PHYSICS_TIMESTEP_NSECS  (SECONDS(1) / 60)
#define PHYSICS_TIMESTEP_SECS   NS_TO_SECONDS(PHYSICS_TIMESTEP_NSECS)
#define PHYSICS_MAX_TIMESTEPS_PER_FRAME 4
#define PHYSICS_MAX_CONTACTS_PER_ENTITY 16

#define WINDOW_W      800
#define WINDOW_H      600
```

```
// keep macros for conditional compilation!
#define ENABLE_DIAGNOSTICS

struct EngineConfig
{
    // selected by the user (public, shown in UI)
    int texture_pixels_per_unit      = 128;
    int camera_pixels_per_unit      = 32;
    int physics_max_timesteps_per_frame = 4;
    int physics_max_contacts_per_entity = 16;
    int window_w                    = 800;
    int window_h                    = 600;
    float target_framerate_secs      = 0.01667f;
    float target_framerate_physics_secs = 0.01667f;

    // derived (hidden, computed at runtime)
    float physics_timestep_secs;
};
```

Strings as resources

```
// button
{
    ITU_EntityId id = itu_entity_create();
    itu_entity_set_debug_name(id, "UI-Button");

    // [...]

    const char btn_text[] = "I am a button!";
    EX6_ImageButton imagebutton = { 0 };
    imagebutton.fn_callback_click = TMP_btn_callback_click;
    imagebutton.ttf_text = TTF_CreateText(
        ttf_engine,
        font_bold,
        btn_text,
        SDL_strlen(btn_text)
    );
}
```

```
// button
{
    ITU_EntityId id = itu_entity_create();
    itu_entity_set_debug_name(id, "UI-Button");

    // [...]

    EX6_ImageButton btn = { 0 };
    imagebutton.fn_callback_click = TMP_btn_callback_click;
    btn.id_string =
        itu_sys_rstorage_string_get("lbl_game_start");
}

// itu_sys_ui.cpp
TTF_Text* get_rendered_text(ITU_IdString id, va_list vlist)
{
    char txt_buffer[MAX_STRING_SIZE];
    char* fmt = itu_sys_rstorage_string_get_ptr(id);

    // variadic string printf: formats arguments based on
    // fmt, and stores the result in txt_buffer
    vsprintf(txt_buffer, fmt, vlist);

    return TTF_CreateText(
        ttf_engine,
        font,
        txt_buffer,
        strlen(txt_buffer)
    );
}
```